



Instructor Guide — Lesson 7: Launch Your Professional Website

Metadata

- **Path:** Digital Independence Path
- **Course:** Course 16 — Build Your Professional Identity
- **Lesson:** 7 of course — "Launch Your Professional Website"
- **Age range:** 15–18
- **Format:** Live, instructor-led, virtual classroom
- **Duration:** 60 minutes
- **Tools used:** GitHub, Vercel, HTML/CSS (already written), Canva (brand kit reference only), Google Docs (submission log)
- **Prerequisites:** Lesson 3 (portfolio site built locally), Lesson 4/6 (bio + 2–3 case studies written), Lesson 5 (brand kit — colors/fonts applied to the site)
- **Materials students need:** Their local portfolio project folder (HTML/CSS files), a personal email address, a phone or laptop to test the live link, parent/guardian awareness if under 18 per platform terms (see below)

Overview

Every prior lesson in this course produced something real but invisible — files sitting on one laptop. This lesson is the payoff: students take that folder and turn it into a live web address anyone in the world can open. This is the first time in the Digital Independence Path that a student's work leaves their own device and becomes a public, shareable artifact under their name. The lesson has two technical milestones (a GitHub repository holding the code, and a Vercel deployment serving it) and one emotional milestone (seeing their own name and work load in a real browser tab).

Instructor Prep (do before class)

1. **Have your own working GitHub account and Vercel account ready**, logged in, so you can screen-share a live demo rather than describe steps abstractly. Deploy a throwaway test site the week of class to confirm the flow still matches your screenshots/script.
2. **Verify the current sign-up flow for both GitHub and Vercel before every session.** Both companies change their onboarding screens, button labels, and verification steps periodically (email verification methods, CAPTCHA steps, "connect your Git provider" prompts, plan-selection screens). Do not assume this guide's screen-by-

screen description is pixel-perfect on the day you teach — walk through both sign-ups yourself within the same week as class and adjust your talking points/annotations to match what you actually see.

3. **Know the age policy language and state it accurately.** GitHub's Terms of Service have historically required users to be at least the "age of digital consent" in their country (commonly 13, with parental involvement expected below the age where a platform allows independent account holders) — check GitHub's current Terms of Service page before class, since this can change. This course path (Digital Independence Path) is scoped to ages 15–18, so every student in the room already clears any typical 13+ threshold; you do not need to screen for younger students in this lesson. Still, say the age policy out loud (see Talking Points) so students understand *why* it exists and get in the habit of reading terms of service, and note that some students may want to loop in a parent/guardian simply because the account is tied to their real name and identity — that's good practice, not a requirement you're imposing.
4. **Confirm Vercel's free tier claim before class.** As of this writing, Vercel's free "Hobby" tier does not require a credit card to sign up and deploy a personal project. Confirm this is still accurate by checking Vercel's current pricing page before class — pricing/tier policies can change.
5. Have the troubleshooting table (below) open in a side window during class so you can react fast.
6. Decide in advance which path you'll teach as primary: **Git/terminal push** (better long-term skill) or **GitHub web upload** (faster, zero terminal, good fallback for a first class or shorter session). This guide teaches Git/terminal as primary with web upload flagged as the no-terminal alternative at each relevant step.

Step-by-Step Teaching Sequence (60 minutes)

1. Hook & Framing (5 min)

- Open with a question: "Right now, how would someone else see your website?" Let a couple of students answer (usually: "email them the file," "screen share it"). Point out the problem: a zip file or local folder is not a *link*. A recruiter, college admissions reader, or client can't click a zip file from their phone in three seconds — but they will click a link.
- State the goal: "By the end of this hour, every one of you will have a real web address — yourname.vercel.app or similar — that works on any device, anywhere, and stays online after you close your laptop."

2. Plain-Language Concepts (5 min)

- Introduce three terms with the moving-boxes analogy (full version is in the Student Lesson — reference it here, don't re-derive):
- **Git** = the packing system that tracks every version of your boxes.
- **GitHub** = the storage warehouse online where your labeled boxes live so others can find them.
- **Vercel** = the shop window that takes what's in the warehouse and displays it live to the public, updating automatically whenever the warehouse changes.
- Say explicitly: "You already did the hard creative work — the bio, case studies, and design. Today is packaging and shipping, not building."

3. Create GitHub Account (5–7 min)

- Live demo: github.com → Sign up → email, username, password/verification.
- Talking point: "Pick a username you'd be okay with a future employer seeing — like `jriviera-dev`, not a gamer tag." This account is often permanent and searchable.
- State the age-policy note from Prep #3 here, briefly and factually — do not dwell.
- Students complete signup and confirm their email in the background while you move to the next step (email confirmation often has a short delay).

4. Create a Repository (5 min)

- Demo: click "New repository" → name it (e.g., `my-portfolio` or `firstname-portfolio`) → set to **Public** (required for the free Vercel flow to easily detect it, and the point is for people to see it) → do NOT initialize with a README if they already have local files (avoids merge conflicts) — OR initialize with README if they're using the web-upload fallback.
- Clarify "public repo" is not the same as "public site yet" — the code being visible on GitHub is a separate thing from the site being live; that's the next step.

5. Push Code to GitHub (10–12 min)

Primary path — Git/terminal:

```
cd my-portfolio-folder
git init
git add .
git commit -m "Initial portfolio site"
git branch -M main
git remote add origin https://github.com/USERNAME/REPO-NAME.git
git push -u origin main
```

- Walk through each line's purpose in one sentence as you type it: `init` starts tracking, `add .` stages everything, `commit` saves a labeled snapshot, `push` uploads it.
- Students will likely need to authenticate (a browser popup or a personal access token, depending on current GitHub flow — verify which is current before class per Prep #2).

No-terminal fallback — GitHub web upload: - On the repo page, click "Add file" → "Upload files" → drag the entire project folder's contents in → commit directly on the `main` branch. - Flag clearly: this works fine for this lesson but students lose the version-history habit `git commit` builds — mention it's a trade-off, not a lesser choice for today.

6. Connect Vercel & Deploy (10 min)

- Demo: vercel.com → Sign up/log in **using their GitHub account** (this is the fastest path and links the two services immediately) → "Add New Project" → select the repository just pushed → Vercel auto-detects a static HTML/CSS site → click **Deploy**.
- Narrate what's happening during the ~30–60 second build: "Vercel is taking the exact files sitting in your GitHub warehouse and putting them on a public server right now."

- Reassure: no credit card should be requested for this free tier — if a student is prompted for payment info, stop and check with them individually; verify current Vercel policy per Prep #4 rather than promising this in absolute terms.

7. Verify the Live URL (5 min)

- Every student clicks their new `*.vercel.app` link.
- Have them check on **two devices** if possible (laptop + phone) — this catches responsive-design issues from earlier lessons and confirms it's truly public, not just visible on their own network.
- Have them send their live link to a partner in the class (chat/breakout) so someone *other than themselves* opens it — the real test of "is this actually live for the world."

8. Common Misconceptions to Address (fold in throughout, or as a mini-recap, 3–5 min)

- **"Deploying is permanent and scary — what if I break it?"** Correct this directly: deploying is not permanent or risky. Every push creates a new deployment; nothing is deleted, and undoing a bad change is just pushing a fix or reverting to a previous commit. Encourage experimentation.
- **"I need to buy a domain name to have a real website."** Not true for this lesson. The `*.vercel.app` address is a fully real, working, public URL — plenty professional for a student portfolio. A custom domain (like `firstname.com`) is a nice future upgrade, entirely optional, and out of scope today — mention it exists, don't teach it.
- **"If I make a mistake, I have to start over."** Address with the git/version-history framing: every commit is a saved checkpoint.

9. Lesson Close (3–5 min)

- Recap the pipeline in one line: "Local files → committed to Git → pushed to GitHub → deployed by Vercel → live URL."
- Assign submission: each student submits their live Vercel URL and their GitHub repo URL (see Student Lesson submission instructions).
- Preview next lesson if applicable (e.g., ongoing updates/maintenance, or moving to custom domains as an elective extension).

Student Difficulties & Troubleshooting Table

Problem	Likely Cause	Fix
<code>git: command not found</code>	Git not installed on this machine	Use the GitHub web-upload fallback instead of debugging installs mid-class
<code>git push</code> asks for password and rejects it	GitHub no longer accepts plain passwords for git operations in most current flows	Use the browser-based sign-in popup if offered, or generate a personal access token per GitHub's current instructions (verify current method before class)

Problem	Likely Cause	Fix
"fatal: not a git repository"	Ran <code>git add / commit</code> before <code>git init</code> , or ran command in wrong folder	<code>cd</code> into the correct project folder first; confirm with <code>ls</code> that HTML files are present, then <code>git init</code>
Merge conflict / "failed to push, updates were rejected"	Repo was initialized with a README on GitHub while local folder also has commits, causing divergent histories	Simplest fix for this lesson: do not initialize with a README when creating the repo if pushing local code; if it already happened, use <code>git pull --allow-unrelated-histories</code> then resolve, or start a fresh repo — don't let one student's merge conflict eat class time
Vercel build fails / shows an error page	Wrong root directory selected, or an index.html isn't at the expected top level of the repo	Check the project's root — index.html should be at the repo root (or the folder Vercel is told to build from) — adjust the project's root directory setting in Vercel and redeploy
Site deploys but shows blank/broken styling	CSS file path is wrong (case-sensitive path, or a local absolute file path was hardcoded)	Check that <code><link></code> tags use relative paths matching the actual repo file names/casing exactly
Student can't remember GitHub/Vercel password mid-class	New account, not saved anywhere	Use the password manager / "forgot password" flow immediately rather than creating a second account
Live link works for the student but not for a partner testing it	Student is testing a local file path (file://) rather than the deployed vercel.app URL	Confirm they copy-pasted the actual <code>https://...vercel.app</code> address, not a local path

How to Review

For each student, confirm: 1. The **GitHub repo URL** opens and shows their actual project files (not an empty repo). 2. The **live Vercel URL** actually loads in a browser — click it yourself, don't take their word for it. 3. The live site reflects their brand kit (Lesson 5) and includes their bio and case studies (Lesson 4/6) — i.e., it's their real, finished content, not a placeholder template. 4. The site loads correctly on at least one other device/browser than the one used to build it (mobile responsiveness check).

A student is not "done" until both URLs are collected and both have been clicked and verified by you or a peer — a link that only works on the author's machine has not actually launched.